

Lake District Trip Planner - Architecture

A capstone project for the Digital Futures programme. Helps users plan a hiking trip in the Lake District through a natural-language chat interface that proposes a walk, picks an accommodation near the trailhead, and shows both on an interactive map.

This document covers everything: scraping, data, backend, frontend, and deployment.

1. What the system does, end-to-end

1. The user opens the site and types a request like "I want a moderate two-day hike near Keswick".
2. The frontend sends that message to `POST /api/chat`.
3. The backend's agent loop calls an LLM (Claude Haiku via OpenRouter). The LLM has access to six tools: `search_walks`, `get_walk_details`, `search_accommodation`, `update_trip_state`, `get_trip_state`, `show_on_map`.
4. The LLM filters the walk catalogue, picks an accommodation routed to the trailhead, writes the choice into the session's structured trip state, and calls `show_on_map`.
5. The frontend polls `/api/sessions/{id}/state` every 2 seconds, and when the `map.walk_ids` or `map.accommodation_ids` change it fetches the GPX GeoJSON and re-renders the Leaflet layers.
6. Reply text streams back over Server-Sent Events and renders bubble-by-bubble in the chat panel.

All of this runs in one FastAPI process on Google Cloud Run.

2. High-level architecture

```
+-----+ | Browser (templates/index.html) | | | | -  
Leaflet map | | - Chat panel (SSE consumer) | | - Polls /api/sessions/{id}/state  
| +-----+ | | HTTPS v  
+-----+ | FastAPI app (server/main.py) | | -  
SSE-streamed agent loop | | - Per-session state + history | | - Static + GPX +  
walk JSON APIs | +---+-----+-----+ | | | v v v +-----+  
+-----+ +-----+ | LLM | | OSMnx | | Filesystem data | | via |  
| walk/drive | | walks.csv | | OR | | graphs | | accommodation | +-----+  
+-----+ | gpx/*.gpx | +-----+
```

3. The data layer

The runtime knows about three datasets, all built once during development and shipped inside the container.

3.1 Walks (CSV + GPX folder)

Source: Walking Britain ([walkingbritain.co.uk](https://www.walkingbritain.co.uk)).

Scraper: `Scraper.py`.

Scraping is a two-stage crawl with caching, run from the project root with `python Scraper.py`.

1. **Listing page** at <https://www.walkingbritain.co.uk/Lake-District-walks> gives walk IDs, areas, grades, and which walks have GPX files. Cached to `cache/lake_district_walks.html`.
2. **Per-walk page** at <https://www.walkingbritain.co.uk/walk-{{ID}}-description> gives full metadata: title, Wainwrights, county, grade, distance, ascent, estimated time, start lat/lng, postcode, full route description. Each page is cached individually under `cache/walks/walk_{{ID}}.html`.
3. **GPX file** at <https://www.walkingbritain.co.uk/download.php?id={{ID}}> when available, saved to `output/gpx/walk_{{ID}}.gpx`. Walking Britain's listing page has a star marker that *should* mean a GPX exists, but the download is sometimes a 1-byte empty file. Two walks (1158 and 2005) are in the `EXCLUDED_WALK_IDS` set for this reason.
4. **Output:** `output/walks.csv` (~219 rows, one per walk that has a usable GPX) plus `output/gpx/` containing one `.gpx` file per row.

The scraper sleeps `REQUEST_DELAY_SECONDS = 1.5` between requests and uses a descriptive User-Agent so it is polite to the source site.

3.2 Accommodation (JSON)

Source: OpenStreetMap, queried through the Overpass API.

Fetcher: `fetch_accommodation.py`.

A single Overpass query selects every node and way inside the Lake District National Park boundary tagged with one of: `hotel`, `guest_house`, `hostel`, `camp_site`, `inn`, `apartment`, `caravan_site`. The flattener pulls out the tags the app actually uses (name, lat/lon, address, phone, website, brand) and writes them to `output/accommodation.json` (~870 entries). At runtime each entry gets an `osm_id` field of the form `node/302866833` so the LLM can pass a stable identifier back through `show_on_map`.

`fetch_all_places.py` is a more general variant that fetches every interesting POI in a bounding box; not used by the live app but kept around for exploration.

3.3 Routing graphs (GraphML)

Source: OpenStreetMap, fetched via `osmnx`.

Builders: `build_walking_graph.py`, `build_driving_graph.py`.

Both build scripts call `osmnx.graph_from_bbox` over the Lake District bounding box (`W=-3.45`, `S=54.35`, `E=-2.75`, `N=54.80`) and save the result as GraphML.

- **Walking graph** (`network_type="walk"`) covers footpaths, bridleways and roads usable on foot. ~50 MB. Loads in ~8 s.
- **Driving graph** (`network_type="drive"`) covers drivable roads only. ~8 MB. Loads in ~2 s.

These graphs are how the app converts crow-flies distances into realistic "how far is the hotel from the trailhead by this transport mode" numbers.

The graphs are pre-warmed at server startup (see `server/main.py` `lifespan`) so the first user request does not pay the 10 s load cost.

4. The backend

Single FastAPI application under `server/`. Eight files, each with one job.

4.1 `server/main.py` - HTTP entry point

Defines the FastAPI app and all routes:

Route	Purpose
GET <code>/</code>	Serves templates/index.html
GET <code>/healthz</code>	Liveness probe
POST <code>/api/chat</code>	Streams an agent turn as SSE
GET <code>/api/sessions/{id}/state</code>	Returns current trip state (used for map polling)
DELETE <code>/api/sessions/{id}</code>	Clears state and conversation history
GET <code>/api/walks</code>	Lists walks with <code>gpx_available=true</code>
GET <code>/api/accommodation</code>	Lists all accommodation
GET <code>/api/walks/{id}/gpx</code>	Returns one walk as a GeoJSON LineString

The `lifespan` context manager pre-warms both routing graphs at startup.

`POST /api/chat` returns a `StreamingResponse` of SSE events. The body of `event_stream()` translates events from the agent loop into SSE frames and writes the final history back into the session store on a clean completion.

4.2 `server/llm.py` - LLM client and agent loop

The brain of the app.

- Configures the OpenAI SDK against OpenRouter (`base_url="https://openrouter.ai/api/v1"`).
- Loads `OPENROUTER_API_KEY` and `MODEL` from `.env` via `python-dotenv`.
- Holds the system prompt. The system prompt is intentionally long because it encodes hard rules ("never invent numbers", "always call `show_on_map` after changing the proposal"), reply style, tool order of operations, named-place coordinates for proximity searches, walk-grade definitions, accommodation reasoning, transport-mode handling, and beginner-safety nudges.
- Defines the JSON schemas for the six tools (`TOOL_SCHEMAS`).
- Exposes `_build_tool_handlers(session_id)` which returns the actual Python callables (state and map tools get the session id closed over so the LLM never sees it).
- `run_agent_stream` is the agent loop: yield text deltas, dispatch tool calls, feed tool results back into the conversation, repeat up to `MAX_AGENT_ITERATIONS = 10`.
- Holds per-session conversation history in `_conversation_history`.

There is also a deterministic safety net: if the model researched in a turn (called `search_walks` or `search_accommodation`) but did not call `show_on_map`, the loop synthesises a `show_on_map` call from the most recent search results so the map cannot go stale. This is the auto-show block. A synthetic `tool_use` assistant message is appended before the synthetic `tool_result` so strict providers (Anthropic via Bedrock) accept the conversation.

4.3 `server/tools.py` - The Python side of the LLM tools

Three of the six tools live here (the other three are state/map closures built in `llm.py`).

- `search_walks` - structured filter over `output/walks.csv`. Supports query substring match, difficulty filter, distance/ascent thresholds, area fuzzy match (`rapidfuzz`, cutoff 80), and proximity search using Haversine distance from a (`near_lat`, `near_lon`, `radius_km`) triple. Returns lightweight summaries.
- `get_walk_details` - returns the full row for a single walk, including the long description.
- `search_accommodation` - two-phase. Phase 1 is a crow-flies filter that picks the candidates inside `radius_km`. Phase 2 routes each candidate to the anchor point through the OSMnx walking or driving graph and sorts by the routed distance. Falls back to crow-flies if a particular routing query fails. Each result includes both the routed and crow-flies distance for transparency.

4.4 `server/routing.py` - OSMnx wrapper

- Lazily loads the walking or driving graph on first use, caches it for the lifetime of the process.
- `route_between` snaps the two lat/lon inputs to the nearest graph node and runs `nx.shortest_path` with the OSM `length` attribute as the weight. Edge geometries are extracted from the Shapely `LineString` stored on each edge so the returned polyline follows real road curves rather than just joining intersections with straight lines.
- `route_highway_breakdown` groups route distance by OSM `highway` tag, used during development to confirm walking routes favour `footway/path` and driving routes use

`primary/secondary/residential`.

- `prewarm()` is called from the FastAPI `lifespan` so neither graph load hits the first user.

4.5 `server/state.py` - Trip state and per-session store

Pydantic models with `extra="forbid"` so any LLM typo in a key surfaces as a validation error instead of corrupting state silently.

The state shape:

```
TripState +-- preferences (Preferences: difficulty, distance/ascent, areas, ...)
+-- group (Group: size, composition) +-- logistics (Logistics: num_days,
transport_mode, accommodation_types, ...) +-- plan (Plan: list[PlannedDay]) +--
status ("gathering" | "proposing" | "confirmed" | "iterating") +-- open_questions
(list[str]) +-- future_considerations (trip_purpose, trip_dates) +-- map
(MapState: walk_ids, accommodation_ids, fit_bounds)
```

`update_trip_state` deep-merges nested dicts (lists are *replaced*, not appended) and re-validates the result against `TripState`.

Sessions live in `_sessions: dict[str, TripState]` keyed by UUID. This is in-memory only; a process restart loses every conversation. The frontend treats an empty state as "no map update yet" rather than as "clear the map" to soften the impact of a restart.

4.6 `server/data.py` - File-backed loaders

Two `lru_cache`-wrapped functions: `load_walks()` parses `output/walks.csv` into a list of dicts with normalised numeric types, and `load_accommodation()` loads the JSON and adds the synthetic `osm_id` field. Both run once per process at first call.

4.7 `server/gpx.py` - GPX parsing helpers

`load_gpx_points(walk_id)` parses an `output/gpx/walk_{ID}.gpx` file into a list of `(lat, lon)` tuples, autodetecting the GPX namespace and falling back from `<rtept>` to `<trkpt>`. There is also a `find_join_point` helper that classifies a route as circular vs linear and picks the nearest route point to a given accommodation, used in earlier prototypes for the "connector line" feature that is no longer part of the live app.

4.8 `server/cli.py`

A small `python -m server.cli` wrapper that calls `run_agent` in a REPL. Useful for testing the agent loop without spinning up the web server.

5. The frontend

Single page: `templates/index.html`. Uses Leaflet and the OpenStreetMap tile layer; no build step, no JS framework.

5.1 Map rendering

On load: - Pre-fetches the entire accommodation list once into a lookup by `osm_id`, so the LLM's accommodation suggestions can render markers instantly without a per-suggestion fetch. - Fetches each chosen walk's GPX as a GeoJSON LineString from `/api/walks/{id}/gpx` and adds it as a styled `L.geoJSON` layer.

A 2-second polling loop (`setInterval(_pollMapState, 2000)`) reads `/api/sessions/{id}/state`, fingerprints the `map` field plus `logistics.transport_mode`, and only re-renders when something changed. A generation counter (`_mapGen`) cancels stale async fetches so a fast sequence of agent updates does not double-render.

Scaling note. Polling works inside one process. For multi-instance Cloud Run the map state would need to live in a shared store (Redis, Firestore) or the channel would need to be SSE/WebSocket. For now `--max-instances 1` keeps everyone on one box.

5.2 Chat rendering

POST `/api/chat` returns SSE. The frontend reads the stream chunk by chunk and dispatches by event type:

Event	Effect
<code>session</code>	Pin <code>sessionId</code> so the map poll can target the right state
<code>tool_call</code>	Show a friendly status line ("Searching walks...") in the loading bubble
<code>text</code>	Append the delta to the bubble, re-render via a tiny Markdown helper
<code>done</code>	Stop reading
<code>error</code>	Replace bubble text with the error

The Markdown renderer handles only `bold`, `italic`, ``code`` and newlines, after escaping HTML.

The "New conversation" button clears the map layers locally, calls `DELETE /api/sessions/{id}` on the server, and resets the chat panel.

6. LLM integration details

- **Provider:** OpenRouter's OpenAI-compatible endpoint. The OpenAI SDK is pointed at `https://openrouter.ai/api/v1`.
- **Model:** `anthropic/claude-haiku-4-5-20251001` set via `MODEL` in `.env`.
- **Optional fallback chain:** `MODEL_FALLBACKS` in `.env` (comma-separated) is forwarded as `extra_body.models` so OpenRouter can fail over to the next model on a rate limit or outage.
- **Token cap:** `MODEL_MAX_TOKENS` defaults to 4096.
- **Tool calling:** OpenAI Chat Completions tool format. The agent loop accumulates streaming `tool_call` deltas by their `index` field (the SDK guarantees stable indices per call), then dispatches them in order.
- **History:** Each `done` event from the agent loop returns the full message history; `event_stream` writes that back into `_conversation_history[session_id]` only on a clean completion, so a failed turn does not poison the next request.

7. Deployment

7.1 Image: Dockerfile

`python:3.11-slim-bookworm`, `pip-installs requirements.txt`, copies `server/`, `templates/`, and the runtime data:

- `output/walks.csv`
- `output/accommodation.json`
- both `.graphml` graph files
- `output/gpx/`

The Cloud Run-injected `$PORT` is honoured at runtime via the shell-form CMD `exec uvicorn server.main:app --host 0.0.0.0 --port ${PORT}`.

7.2 Build and deploy: `deploy.ps1`

Idempotent end-to-end deploy. Steps:

1. `gcloud config set project hiking-trip-planner-495520`.
2. Ensure the Secret Manager secret `openrouter-api-key` has at least one version. On the very first deploy it reads `OPENROUTER_API_KEY` from `.env` and uploads it as the first version.
3. Grant the Cloud Run runtime service account (`{project_number}-compute@developer.gserviceaccount.com`) the `roles/secretmanager.secretAccessor` role on that secret.
4. `gcloud run deploy lake-district-planner --source . --region europe-west2` with `--memory 2Gi --cpu 2 --min-instances 0 --max-instances 1 --timeout 300 --update-secrets="OPENROUTER_API_KEY=openrouter-api-key:latest"`.

The build is **Cloud Build, not local Docker** (`--source` uploads the project and Google builds the image server-side), so you do not need Docker installed locally.

7.3 Why these flags

- `min-instances=0`: free when idle. First request after idle pays ~10 s for the cold start (graph loading dominates).
- `max-instances=1`: required because trip state and conversation history are in-process. Two instances would not see each other's sessions.
- `memory=2Gi`: the walking graph alone is ~50 MB on disk and inflates to a few hundred MB in memory once loaded.
- `timeout=300`: an agent turn with multiple tool calls can take 30-90 s on Haiku; Cloud Run's default 60 s would cut some of those off.

7.4 Files explicitly included for deployment

`.gcloudignore` overrides `.gitignore` for the deploy upload. The graphml files live in `output/` which `.gitignore` excludes, but `.gcloudignore` re-includes them so they make it into the image.

7.5 Live URL

<https://lake-district-planner-644255749905.europe-west2.run.app>

8. Testing

`tests/` holds unit and integration tests:

- `test_state.py` - Pydantic state model: deep merge, validation, list replacement semantics.
- `test_tools.py` - The three filter tools: covers query, difficulty, proximity, area fuzzy match, accommodation type filter, routed-distance ordering.
- `test_api_routes.py` - FastAPI route plumbing (TestClient). Smoke-tests every route except `/api/chat`, which is exercised manually via the UI to avoid spending tokens on every test run.
- `test_llm_imports.py` - sanity check that `server/llm.py` imports cleanly.
- `test_llm_behaviour.py` - end-to-end agent behaviour against a real LLM. Run on demand only; uses real tokens.

Run from the project root: `python -m tests.test_<name>`.

9. Local development

`run.ps1` is the one-shot bootstrap:

1. Verifies Python is available.
2. Creates `.venv` if missing and installs `requirements.txt`.
3. Warns if `.env` is missing.
4. Builds the walking and driving graphs if absent (one-off ~3 minute download from OSM).
5. Launches `uvicorn server.main:app --host 0.0.0.0 --port 8000`.

Subsequent runs are seconds. The browser opens at `http://localhost:8000`.

10. Repository layout (top level)

```
Capstone_Project/ +-- CLAUDE.md project brief / agent instructions +--
ARCHITECTURE.md this file +-- requirements.txt +-- Dockerfile +-- run.ps1 local
launcher +-- deploy.ps1 Cloud Run deploy +-- .gcloudignore overrides .gitignore
for the deploy upload +-- Scraper.py Walking Britain scraper +--
fetch_accommodation.py OSM Overpass accommodation fetcher +--
build_walking_graph.py OSMnx walking graph builder +-- build_driving_graph.py
OSMnx driving graph builder +-- cache/ raw HTML responses (not deployed) +--
output/ | +-- walks.csv ~219 walks | +-- accommodation.json ~870 places | +--
gpx/ one .gpx per walk_id | +-- lake_district_walking_graph.graphml | +--
lake_district_driving_graph.graphml +-- server/ FastAPI backend | +-- main.py
HTTP routes, SSE, lifespan | +-- llm.py agent loop, system prompt, tool schemas |
+-- tools.py search_walks / get_walk_details / search_accommodation | +--
state.py Pydantic trip state, session store | +-- routing.py OSMnx walking and
driving routing | +-- gpx.py GPX parsing | +-- data.py cached CSV/JSON loaders |
+-- cli.py REPL for the agent loop +-- templates/ | +-- index.html single-page
UI: Leaflet + chat + SSE +-- tests/ +-- test_state.py +-- test_tools.py +--
test_api_routes.py +-- test_llm_imports.py +-- test_llm_behaviour.py
```

11. Limits and known sharp edges

- **In-memory state.** A Cloud Run cold start wipes every session. Acceptable for a single-user demo. A persistent store (Redis, Firestore, SQLite via a Cloud Run Volume) is the obvious next step.
- **Single-instance only.** `max-instances=1` is a load-bearing assumption for the polling loop and the in-memory session dict. Multi-instance scaling would need the state moved out and the map channel changed to SSE/WebSocket fed by a shared store.
- **No retries on the LLM stream.** A network blip mid-stream surfaces as an `error` event in the chat. The user can resend.
- **GPX coverage.** Only walks with a real (non-empty) GPX from Walking Britain are in `walks.csv`; ~219 of the listed 220-odd Lake District walks. Two were excluded by `walk_id` (1158, 2005). Walks listed on the source site without a GPX never make it into the dataset.
- **Lake District only.** Hard-coded bounding box, area names, and named- place coordinates. Extending to Snowdonia or Scotland would mean a new scrape, new graphs, and a longer system

prompt.